



中國人民大學
RENMIN UNIVERSITY OF CHINA



目录

CONTENTS

01 模型选用 PART

02 模型调整 PART

03 程序加速 PART

张海源 2023200249

▪ 尝试**堆叠法 (Stacking)**，加入LightGBM、CatBoost和原有模型作为基模型，Ridge回归为元模型学习权重

简单的加权或等权均值表现较差 (30分左右)
采用OOF预测防止数据泄露
集各家所长、有效降低RMSE、结果更稳定

▪ 使用**bert-base-chinese**分析“房屋优势”“核心卖点”“户型介绍”“周边配套”“交通出行”“客户反馈”。
将输出向量作为特征

模型	In-Sample RMSE	Out-of-Sample RMSE	6-fold CV RMSE	Kaggle
OLS	522973.08	523296.03	522237.41	66.8
	208657.76	205091.17	208029.64	
Random Forest	99904.81	267618.07	261349.21	71.5
	53552.19	135043.67	132892.08	
ANN	465143.67	465906.93	599178.16	66.9
	192047.91	194922.84	209142.44	
XGBoost	156312.89	238582.06	235936.44	72.8
	84692.08	128911.70	127126.55	
LightGBM	214551.93	255234.89	256294.73	72.7
	109956.59	131998.43	131329.69	
CatBoost	248595.25	274973.40	276031.35	72.4
	130606.70	141718.81	142674.96	
Stacking			233525.52 125351.31	73.1

(加入BERT)
Price
Rent

模型	In-Sample RMSE	Out-of-Sample RMSE	6-fold CV RMSE	Kaggle
OLS	522973.08	523296.03	522237.41	66.7
	208657.76	205091.17	208029.64	
Random Forest	99904.81	267618.07	261349.21	72.4
	53552.19	135043.67	132892.08	
ANN	465143.67	465906.93	599178.16	66.9
	192047.91	194922.84	209142.44	
XGBoost	156312.89	238582.06	235936.44	74.1
	84692.08	128911.70	127126.55	
LightGBM	214551.93	255234.89	256294.73	73.3
	109956.59	131998.43	131329.69	
CatBoost	248595.25	274973.40	276031.35	72.7
	130606.70	141718.81	142674.96	
Stacking			233525.52 125351.31	74.0

(去除BERT)
Price
Rent

02 模型调整

PART



中國人民大學
RENMIN UNIVERSITY OF CHINA

参数调整

```
XGBRegressor(n_estimators=3000, learning_rate=0.008, max_depth=9, subsample=0.7
```

```
colsample_bytree=0.7, reg_alpha=0.1, reg_lambda=1.0, n_jobs=-1,
```

函数调整

- MAE的使用 (X) `objective='reg:absoluteerror'`
- ANN各层调整
- bert各层调整

```
self.model = nn.Sequential(  
    nn.Linear(self.input_dim, self.hidden_dim),  
    nn.BatchNorm1d(self.hidden_dim),  
    nn.ReLU(),  
    nn.Dropout(0.2),  
    nn.Linear(self.hidden_dim, self.hidden_dim // 2),  
    nn.BatchNorm1d(self.hidden_dim // 2),  
    nn.ReLU(),  
    nn.Linear(self.hidden_dim // 2, 1)  
) .to(self.device)
```

```
def __init__(self, model_name='bert-base-chinese', n_components=16, batch_size=32, whiten=False):  
    self.device = torch.device("cuda" if torch.cuda.is_available() else "cpu")  
    self.pca = PCA(n_components=n_components, whiten=whiten)  
    self.is_fitted = False  
    self.batch_size = batch_size  
    self.model_name = model_name  
    self.max_length = 128
```

03 程序加速

PART



中國人民大學
RENMIN UNIVERSITY OF CHINA

torch.compile和triton的引进

```
model.eval()
if device.type == 'cuda':
    # 编译模型以加速
    try:
        model = torch.compile(model, mode='default')
    except Exception as e:
        print(f"BERT compile warning: {e}")
return tokenizer, model
```

```
@triton.jit
def mse_kernel(
    output_ptr, target_ptr, loss_ptr,
    n_elements,
    BLOCK_SIZE: tl.constexpr,
):
```

```
pid = tl.program_id(axis=0)
block_start = pid * BLOCK_SIZE
offsets = block_start + tl.arange(0, BLOCK_SIZE)
mask = offsets < n_elements
output = tl.load(output_ptr + offsets, mask=mask)
target = tl.load(target_ptr + offsets, mask=mask)
diff = output - target
loss = diff * diff
tl.store(loss_ptr + offsets, loss, mask=mask)
```

图捕获、图优化

优化前的LayerNorm + Linear

```
hidden_states = layer_norm(hidden_states) # LayerNorm
hidden_states = linear(hidden_states)      # 全连接层
```

优化后: 融合LayerNorm和Linear

数学等价性:

$\text{LayerNorm}(x) = \gamma * (x - \mu) / \sigma + \beta$

融合后:

$\text{Linear}(\text{LayerNorm}(x)) = W * (\gamma * (x - \mu) / \sigma + \beta) + b$

合并参数:

$W' = W * \gamma / \sigma, b' = W * \beta + b - W * \mu / \sigma$

变为单个操作:

$W'x + b'$

bert的缓存

```
if cache_path.exists():
    try:
        data = np.load(cache_path, allow_pickle=False)
        cached = data["features"]
        if cached.shape[0] == len(df) and cached.shape[1] == n_components:
            all_features = cached
            if verbose: print(f"Loaded BERT features from cache: {cache_path}")
        except Exception:
            all_features = None
    np.savez_compressed(cache_path, features=all_features)
```

数据头尾 + 参数 → 哈希值名储存